LeafyCV1.0

Final Frontend + Backend + Database Integration Handover Guide

Purpose: This document is the final connection contract for the Computer Vision subsystem. It tells the React, Django/backend and database work how they connect to LeafyCV1.0. It is designed to match the role-specific update briefs already provided to the team.

1. Final system behaviour

The final system must separate the live camera display from saved Computer Vision measurements. The user can watch the camera continuously, but a new historical CV sample is produced only every 30 minutes (or when a deliberate manual sample is requested).

```
LIVE CAMERA PREVIEW
        |
        | transient image only
        | no CameraImage / ImageAnalysis write
        |
        +------ every 30 minutes ------> ONE CV SAMPLE
                                          |
                                          +-> CameraImage
                                          +-> ImageAnalysis
                                          +-> PlantVisionMeasurement per SLOT
```

Why: React needs a live farm view, but running/saving CV on every preview frame would waste CPU/storage and would make growth history noisy. The timestamp shown with the CV result must therefore be the timestamp of the latest saved analysis, not the current live frame.

2. Final ownership and connection map

Component	Owns	Connection to LeafyCV1.0
React frontend	Live camera UI, ROI/SLOT/calibration editing, latest result display	Calls Django APIs only. React does not run OpenCV.
Django backend	API, validation, preview endpoint, image/result persistence	Provides current camera configuration to LeafyCV and accepts scheduled CV samples.
Database	CAMERA config, CAMERA_SLOT_POSITION, images and analysis history	Acts as the shared source of truth for the integrated system.
LeafyCV1.0	Camera capture for scheduled samples, segmentation, measurement, growth, local recovery outputs	Reads current config from backend and POSTs one saved sample at the analysis interval.

3. Database contract

The final Computer Vision relationship uses the existing physical SLOT table. There is only one SLOT entity. CAMERA_SLOT_POSITION is the mapping that says where an existing physical SLOT appears in a particular camera image.

```
FARMLEVEL
    |
    v
  CAMERA -----------------> CAMERA_SLOT_POSITION --------+
    |                                                    |
    v                                                    v
CAMERAIMAGE                                            SLOT
    |                                                    ^
    v                                                    |
IMAGEANALYSIS                                            |
```

```
    |                                                |
    v                                                |
PLANTVISIONMEASUREMENT ----------------------------------+
```

3.1 CAMERA - required CV fields

Field	Required behaviour	Reason
source_type	RTSP or WEBCAM	LeafyCV chooses the correct camera source.
ip_address	nullable	RTSP camera address.
rtsp_port	nullable, normally 554	RTSP connection.
rtsp_path	nullable	RTSP stream path where required.
webcam_index	nullable integer	Local laptop/USB webcam selection.
lighting_mode	NORMAL or MAGENTA	Selects the correct preprocessing profile.
roi_points	nullable JSON: x,y,width,height	Defines the monitoring area using normalized coordinates.
calibration_start_x/y	nullable normalized values	Start of known-length reference line.
calibration_end_x/y	nullable normalized values	End of known-length reference line.
calibration_length_cm	nullable positive value	Converts pixels to centimetres.
enabled	boolean	Allows a camera to be disabled without deleting it.

3.2 CAMERA_SLOT_POSITION - required new mapping

```
camera_slot_position_id   UUID PK
camera_id                 FK -> CAMERA
slot_id                   FK -> existing SLOT
centre_x                  normalized 0..1
centre_y                  normalized 0..1
created_at
updated_at
UNIQUE(camera_id, slot_id)
```

Why: the same physical SLOT has different image coordinates in different cameras. Coordinates therefore belong to the camera-to-SLOT relationship, not directly to SLOT.

3.3 Existing analysis tables

Table	Final CV requirement
CAMERAIMAGE	One record only for an official scheduled/manual CV sample. Keep camera + timestamp + stored image path/file.
IMAGEANALYSIS	Whole-frame result: canopy coverage, visible count, validity and processing time. CameraImage relationship should support the intended schema.
PLANTVISIONMEASUREMENT	Use a real FK to existing SLOT. Keep canopy, size, crowding, growth, visibility and validity. width_cm, height_cm, green_percentage and yellow_percentage must accept null.

Nullable rule: no calibration stick means centimetre values are unknown, not zero. Under MAGENTA lighting, green/yellow colour values are intentionally unavailable and should remain null.

4. Backend API contract

The Django backend is the integration boundary. React and LeafyCV should not write database tables directly.

Endpoint	Used by	Purpose
GET /api/cv/cameras/	React	Load camera list.
POST /api/cv/cameras/	React	Create a camera.
PATCH /api/cv/cameras/{id}/	React	Update camera source, lighting, ROI and calibration fields.
POST /api/cv/cameras/{id}/test/	React	Test one camera frame.
GET /api/cv/cameras/{id}/preview/	React	Return the current transient/live preview image. No analysis write.
GET /api/cv/cameras/{id}/config/	LeafyCV	Return current database-backed camera configuration.
GET /api/cv/cameras/{id}/slot-positions/	React + LeafyCV	Return registered physical SLOT centres for this camera.
POST/PATCH/DELETE slot-position routes	React	Add, move or remove camera-to-SLOT mappings.
GET /api/cv/cameras/{id}/latest-analysis/	React	Return most recent saved analysis + per-SLOT measurements.
POST /api/cv/ingest/	LeafyCV	Accept one official scheduled/manual CV sample.

4.1 Exact camera config response expected by LeafyCV

```
{
  "camera_id": "CAMERA_UUID",
  "name": "Level 1 Camera 1",
  "source_type": "RTSP",
  "ip_address": "192.168.1.106",
  "rtsp_port": 554,
  "rtsp_path": "/stream1",
  "webcam_index": null,
  "lighting_mode": "MAGENTA",
  "roi_points": {"x":0.10,"y":0.15,"width":0.80,"height":0.70},
  "calibration_start_x": 0.15,
  "calibration_start_y": 0.80,
  "calibration_end_x": 0.55,
  "calibration_end_y": 0.80,
  "calibration_length_cm": 20.0,
  "enabled": true
}
```

The backend should not return the RTSP password. LeafyCV reads RTSP credentials from local environment variables on the LeafyCV machine.

4.2 Exact SLOT-position response expected by LeafyCV

```
[
  {
    "camera_slot_position_id": "POSITION_UUID",
    "slot_id": "SLOT_UUID_P01",
    "slot_label": "P01",
    "centre_x": 0.24,
    "centre_y": 0.43
  },
  {
    "camera_slot_position_id": "POSITION_UUID_2",
    "slot_id": "SLOT_UUID_P02",
    "slot_label": "P02",
    "centre_x": 0.49,
    "centre_y": 0.42
```

```
  }
]
```

4.3 LeafyCV ingest request

LeafyCV sends `multipart/form-data` with a JSON string field named `metadata` and a JPEG file field named `raw_image`.

```
metadata = {
  "model_version": "LeafyCV1.0",
  "camera_id": "CAMERA_UUID",
  "camera_name": "Level 1 Camera 1",
  "captured_at": "2026-08-15T16:30:00+10:00",
  "lighting_mode": "MAGENTA",
  "size_calibrated": true,
  "colour_measurement_valid": false,
  "analysis": {
    "total_canopy_coverage": 48.22,
    "visible_plant_count": 4,
    "measurement_valid": true,
    "processing_time_ms": 181
  },
  "plants": [
    {
      "slot_id": "SLOT_UUID_P01",
      "canopy_coverage": 41.50,
      "width_px": 302.0,
      "height_px": 268.0,
      "width_cm": 15.10,
      "height_cm": 13.40,
      "green_percentage": null,
      "yellow_percentage": null,
      "crowding_percentage": 50.0,
      "growth_change": 1.75,
      "visible": true,
      "measurement_valid": true
    }
  ]
}
raw_image = JPEG file
```

Backend ingest order: resolve CAMERA (prefer camera_id when present), save raw image and CAMERAIMAGE, create IMAGEANALYSIS, validate every SLOT UUID, create all PLANTVISIONMEASUREMENT rows in one transaction, then return HTTP 201/2xx.

5. React frontend connection

React is the visual control surface. It shows the live camera and edits configuration through Django. React must not run LeafyCV and must not treat every preview refresh as a new analysis.

React feature	Backend data/API	Reason
Camera selector/settings	camera list + camera PATCH	Select RTSP/webcam and lighting source.
Live preview	GET preview	Continuous monitoring only.
ROI editor	camera roi_points PATCH	Defines the CV monitoring area.
SLOT marker Add/Move/Remove	slot-position APIs	Maintains stable plant identity by physical SLOT.
Calibration stick Set/Adjust/Remove	camera calibration fields PATCH	Provides pixels-to-cm conversion.
Latest whole-camera result	GET latest-analysis	Displays visible count and total canopy.
Latest per-SLOT table/cards	latest-analysis nested measurements	Displays canopy, size, crowding, growth and visibility.

Display rule: Always show `Last analysed: HH:MM`. The image may be live while the measurements are up to 30 minutes old. If cm values are null, show `Not calibrated`. If green/yellow values are null under MAGENTA, show colour analysis as unavailable rather than zero.

6. LeafyCV1.0 integrated operation

For standalone laptop development, use a local JSON config. For the final team integration, use backend-driven mode so React edits saved in the database are automatically read by LeafyCV.

Integrated one-shot test:

```
.\.venv\Scripts\python.exe main.py run-once `
  --backend-base-url http://127.0.0.1:8000 `
  --camera-id CAMERA_UUID
```

Integrated 30-minute production scheduler:

```
.\.venv\Scripts\python.exe main.py run-loop `
  --backend-base-url http://127.0.0.1:8000 `
  --camera-id CAMERA_UUID `
  --interval-minutes 30
```

Local webcam live monitor with 30-minute saved samples:

```
.\.venv\Scripts\python.exe main.py monitor `
  --config configs\my_webcam.json `
  --interval-minutes 30 `
  --reference-length-cm 20
```

In backend-driven mode LeafyCV refreshes the camera configuration from Django. This means React -> Django/database changes to lighting, ROI, calibration and SLOT positions can be used by the next scheduled LeafyCV sample without maintaining a second manual configuration.

7. End-to-end sequence

1. React loads the CAMERA record and CAMERA_SLOT_POSITION records through Django.

2. React requests the backend preview endpoint repeatedly and draws ROI, SLOT markers and calibration line over the live image.

3. User edits ROI/SLOT/calibration in React and presses Save. Django updates CAMERA / CAMERA_SLOT_POSITION only.

4. LeafyCV scheduled process reaches the next 30-minute sample time.

5. LeafyCV fetches the current camera config and SLOT positions from Django.

6. LeafyCV obtains one fresh frame, runs NORMAL/MAGENTA processing, measures each registered SLOT, calculates growth from the previous valid saved sample, and saves local recovery files.

7. LeafyCV POSTs metadata + raw_image to /api/cv/ingest/.

8. Django atomically creates CAMERAIMAGE -> IMAGEANALYSIS -> PLANTVISIONMEASUREMENT rows.

9. React latest-analysis refresh receives the new saved result and updates the displayed timestamp/metrics while live preview continues independently.

8. Field mapping summary

LeafyCV concept	Database/backend field	React use
Camera source	CAMERA.source_type + RTSP/webcam fields	Camera settings form
Lighting profile	CAMERA.lighting_mode	NORMAL/MAGENTA selector
Monitoring ROI	CAMERA.roi_points	ROI overlay/editor
Calibration stick	CAMERA calibration start/end + length	Reference line editor
Physical plant identity	SLOT.slot_id	SLOT selector/label
SLOT image location	CAMERA_SLOT_POSITION.centre_x/centre_y	Marker overlay/editor
Saved frame	CAMERAIMAGE	Not every preview frame
Whole-frame result	IMAGEANALYSIS	Latest canopy/visible count
Per-SLOT result	PLANTVISIONMEASUREMENT	Per-plant cards/table/history

9. Integration acceptance checklist

[ ] Database: CAMERA contains source, lighting, ROI and calibration fields.

[ ] Database: CAMERA_SLOT_POSITION exists and references the existing SLOT.

[ ] Database: PlantVisionMeasurement uses the real SLOT FK and nullable cm/colour fields.

[ ] Backend: config and slot-position GET endpoints return normalized data in the contract above.

[ ] Backend: preview endpoint returns a transient/live image without creating analysis history.

[ ] Backend: ingest accepts metadata + raw_image and writes all records atomically.

[ ] Backend: latest-analysis returns the most recent saved analysis and its per-SLOT measurements.

[ ] React: live preview is visible and independent from analysis refresh.

[ ] React: ROI, SLOT markers and calibration stick can be saved/cancelled through backend APIs.

[ ] React: Last analysed timestamp is clearly shown.

[ ] LeafyCV: backend-driven run-once successfully reads CAMERA + SLOT positions and sends one sample.

[ ] LeafyCV: final scheduled process runs with a 30-minute interval.

[ ] Full system: one scheduled sample produces exactly 1 CAMERAIMAGE, 1 IMAGEANALYSIS and N per-SLOT measurements.

Final connection target

React shows and edits. Django validates and persists. The database is the integrated configuration/history source of truth. LeafyCV performs the image analysis and sends one official result every 30 minutes.